

SQL: A Practical Guide

Introduction to SQL

Structured Query Language, commonly known as SQL, is the standard language used to communicate with relational database systems. It allows users to create, read, update, and delete data stored in tables. Nearly every modern application that stores structured data relies on SQL or a close variant of it, whether the underlying engine is PostgreSQL, MySQL, SQL Server, Oracle, or SQLite.

SQL is a declarative language. Rather than specifying step by step how to retrieve data, the user describes what result is desired, and the database engine's query planner determines the most efficient way to obtain it. This distinguishes SQL from procedural languages, where the programmer specifies each operation explicitly.

Why SQL Matters

Almost every organization that keeps records - financial transactions, customer profiles, inventory counts, sensor readings - stores that information in a relational database. Being able to query, filter, and summarize this data is a foundational skill for analysts, backend developers, and data scientists alike.

Relational Database Concepts

A relational database organizes data into tables, where each table represents an entity such as customers, orders, or products. Each row in a table is a record, and each column represents an attribute of that record.

Primary and Foreign Keys

A primary key uniquely identifies each row in a table. A foreign key is a column in one table that references the primary key of another table, establishing a relationship between the two. This relational structure is what allows SQL to join data across multiple tables.

Normalization

Normalization is the process of organizing data to reduce redundancy and improve integrity. The most common normal forms are first, second, and third normal form. Each successive form removes a particular type of redundancy - repeating groups, partial dependencies, and transitive dependencies respectively.

- First Normal Form: eliminate repeating groups, ensure atomic values
- Second Normal Form: remove partial dependencies on a composite key
- Third Normal Form: remove transitive dependencies between non-key columns

Core SQL Statements

SELECT

The SELECT statement retrieves data from one or more tables. It can filter rows with WHERE, sort them with ORDER BY, and limit the number of rows returned.

```
SELECT customer_id, order_date, total_amount
FROM orders
WHERE total_amount > 100
ORDER BY order_date DESC
LIMIT 10;
```

INSERT, UPDATE, DELETE

These three statements modify data. INSERT adds new rows, UPDATE changes existing rows, and DELETE removes rows that match a given condition.

```
INSERT INTO customers (name, email)
VALUES ('Asha Rao', 'asha@example.com');

UPDATE customers SET email = 'new@example.com'
WHERE customer_id = 42;

DELETE FROM customers WHERE customer_id = 42;
```

CREATE and ALTER TABLE

The CREATE TABLE statement defines a new table and its columns, along with constraints such as NOT NULL, UNIQUE, and PRIMARY KEY. ALTER TABLE modifies an existing table's structure, for example adding a new column.

```
CREATE TABLE customers (
    customer_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) UNIQUE
);
```

Joins and Set Operations

Joins combine rows from two or more tables based on a related column. The most common join types are INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL OUTER JOIN.

```
SELECT o.order_id, c.name
FROM orders o
INNER JOIN customers c
    ON o.customer_id = c.customer_id;
```

An INNER JOIN returns only rows with matching values in both tables. A LEFT JOIN returns all rows from the left table plus matching rows from the right table, filling in NULL where there is no match. A RIGHT JOIN does the opposite, and a FULL OUTER JOIN returns all rows from both tables.

Set Operations

UNION, INTERSECT, and EXCEPT combine the results of two queries. UNION returns the distinct combined rows, UNION ALL keeps duplicates, INTERSECT returns rows common to both queries, and EXCEPT returns rows present in the first query but not the second.

Aggregation and Grouping

Aggregate functions summarize data across multiple rows. Common aggregate functions include COUNT, SUM, AVG, MIN, and MAX. The GROUP BY clause partitions rows into groups so that an aggregate function can be applied to each group separately.

```
SELECT department, AVG(salary) AS avg_salary
FROM employees
GROUP BY department
HAVING AVG(salary) > 60000;
```

The HAVING clause filters groups after aggregation, whereas WHERE filters individual rows before aggregation. This distinction is a common source of confusion for newcomers to SQL.

Window Functions

Window functions perform calculations across a set of rows related to the current row without collapsing the result into a single row, unlike GROUP BY. Common window functions include ROW_NUMBER, RANK, and running totals using SUM with an OVER clause.

```
SELECT employee_id, salary,
       RANK() OVER (ORDER BY salary DESC) AS salary_rank
FROM employees;
```

Subqueries and Common Table Expressions

A subquery is a query nested inside another query. Subqueries can appear in the SELECT, FROM, or WHERE clause, and can return a single value, a single row, or a full table of results.

```
SELECT name FROM customers
WHERE customer_id IN (
    SELECT customer_id FROM orders
    WHERE total_amount > 500
);
```

Common Table Expressions, introduced with the WITH keyword, allow a query to be broken into named, reusable pieces. This improves readability, especially for complex, multi-step queries, and enables recursive queries for hierarchical data such as organizational charts.

```
WITH high_value_orders AS (
    SELECT customer_id, total_amount
    FROM orders WHERE total_amount > 500
)
SELECT c.name, h.total_amount
FROM customers c
JOIN high_value_orders h ON c.customer_id = h.customer_id;
```

Indexes and Query Performance

An index is a data structure that allows the database engine to locate rows quickly without scanning an entire table. Indexes speed up SELECT queries but add overhead to INSERT, UPDATE, and DELETE operations because the index must be maintained.

Reading Execution Plans

Most database systems provide an EXPLAIN command that shows how a query will be executed, including which indexes are used, the order tables are joined, and estimated row counts. Reading execution plans is essential for diagnosing slow queries.

- Add indexes on columns frequently used in WHERE and JOIN clauses
- Avoid over-indexing tables that receive heavy write traffic
- Prefer covering indexes when a query only needs indexed columns
- Watch for full table scans on large tables in the execution plan

Transactions and Data Integrity

A transaction groups multiple statements into a single unit of work that either fully succeeds or fully fails. This is captured by the ACID properties: atomicity, consistency, isolation, and durability.

```
BEGIN;  
UPDATE accounts SET balance = balance - 100 WHERE id = 1;  
UPDATE accounts SET balance = balance + 100 WHERE id = 2;  
COMMIT;
```

Isolation levels control how concurrent transactions interact with one another. Common levels include Read Uncommitted, Read Committed, Repeatable Read, and Serializable, each offering a different trade-off between consistency and concurrency.

Constraints such as NOT NULL, UNIQUE, CHECK, and foreign key constraints enforce data integrity at the database level, preventing invalid data from ever being stored regardless of which application writes to the database.